Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

5η Εργαστηριακή Άσκηση

Εργασία σε assembly του επεξεργαστή ARM

Μάθημα: Σχεδιασμός Ενσωματωμένων Συστημάτων

Εξάμηνο: 9ο

Ονοματεπώνυμα: Μιχαλακόπουλος Δημήτριος (ΑΜ: 03120229), Γκενάκου Ζωή (ΑΜ: 03120015)

Ερώτημα 1:

Μετατροπή εισόδου από τερματικό.

Σκοπός της πρώτης άσκησης είναι η ανάπτυξη ενός προγράμματος, το οποίο να διαχειρίζεται εισόδους από τον χρήστη μέσω τερματικού και να πραγματοποιεί συγκεκριμένες **μετατροπές** στους χαρακτήρες που εισάγονται. Η είσοδος περιορίζεται σε **μήκος έως 32 χαρακτήρες**, με οποιουσδήποτε επιπλέον χαρακτήρες να απορρίπτονται.

Το πρόγραμμα εφαρμόζει τους εξής κανόνες επεξεργασίας:

1. Οι κεφαλαίοι χαρακτήρες του αγγλικού αλφάβητου μετατρέπονται σε πεζούς και αντίστροφα.
2. Οι αριθμητικοί χαρακτήρες στο εύρος [‘0’, ‘9’] μεταβάλλονται βάσει του εξής μοτίβου:

‘0’ → ‘5’ ‘5’ → ‘0’

‘1’ → ‘6’ ‘6’ → ‘1’

‘2’ → ‘7’ ‘7’ → ‘2’

‘3’ → ‘8’ ‘8’ → ‘3’

‘4’ → ‘9’ ‘9’ → ‘4’

3. Όλοι οι υπόλοιποι χαρακτήρες παραμένουν αμετάβλητοι.

Το πρόγραμμα έχει σχεδιαστεί για συνεχή λειτουργία και τερματίζει μόνο όταν ο χρήστης εισάγει τη συμβολοσειρά ‘Q’ ή ‘q’. Επίσης, η υλοποίησή μας έχει πραγματοποιηθεί κάνοντας χρήση τόσο συναρτήσεων της βιβλιοθήκης της C (π.χ. scanf και printf), όσο και των system calls του Linux (π.χ. read και write) και είχε ως στόχο την σχεδίαση μιας αποδοτικής και συμπαγούς συνάρτησης που εκτελεί τις μετατροπές, διατηρώντας τον κώδικα απλό και κατανοητό.

Ας διαχωρίσουμε το πρόγραμμα **ask1** σε μικρότερα τμήματα για να εξηγήσουμε κάθε μέρος με λεπτομέρεια.

```
main:
    push {ip, lr}

@ Loop Entry Point
loop:
    mov r4, #0          @ Initialize character counter
    mov r5, #10         @ Newline character for output

    @ Print Prompt
    mov r0, #1          @ stdout
    ldr r1, =prompt     @ Address of the prompt in memory
    mov r2, #prompt_len @ Length of the prompt
    mov r7, #4          @ write system call
    swi 0

    @ Read Input
    mov r0, #0          @ stdin
    ldr r1, =input_str  @ Address of input string in memory
    mov r2, #32         @ Read 32 characters + newline
    mov r7, #3          @ read system call
    swi 0

    @ Check for Exit Conditions
    cmp r0, #2          @ Could it be 'q\n' or 'Q\n'?
    bne convert
    ldrb r3, [r1]
    cmp r3, #81         @ Is it 'Q'?
    beq exit_program
    cmp r3, #113        @ Is it 'q'?
    beq exit_program
```

1. **Αρχικοποίηση και Είσοδος στο Βρόχο:**
   - Αποθήκευση των καταχωρητών `ip` και `lr` στην στοίβα.
   - Ο βρόχος ξεκινά από την ετικέτα `loop`.
2. **Εκτύπωση Προτροπής στον Χρήστη:**
   - Η system call `write` εκτυπώνει το μήνυμα προτροπής:

     "Input a string of up to 32 characters (Terminate with Q or q):"
3. **Ανάγνωση Εισόδου:**
   - Η system call `read` διαβάζει έως 32 χαρακτήρες από το `stdin` και τους αποθηκεύει στη μεταβλητή `input_str`.
4. **Έλεγχος για Συνθήκες Τερματισμού:**
   - Αν η εισαγωγή περιέχει μόνο τους χαρακτήρες Q ή q, το πρόγραμμα τερματίζει.

```
convert:
    ldrb r3, [r1]       @ Load current character
    add r4, #1          @ Increment character counter
    cmp r3, #10         @ Is it a newline?
    beq output          @ If newline, go to output

    @ Process Characters
    cmp r3, #48         @ Is it a digit? ('0')
    blt handle_next     @ If less than '0', handle other characters
    cmp r3, #57         @ Is it a digit? ('9')
    ble convert_digit   @ If in '0'-'9', process as digit

    cmp r3, #65         @ Is it uppercase? ('A')
    blt handle_next     @ If less than 'A', handle other characters
    cmp r3, #90         @ Is it uppercase? ('Z')
    ble to_lowercase    @ If in 'A'-'Z', convert to lowercase

    cmp r3, #97         @ Is it lowercase? ('a')
    blt handle_next     @ If less than 'a', handle other characters
    cmp r3, #122        @ Is it lowercase? ('z')
    ble to_uppercase       @ If in 'a'-'z', convert to uppercase

handle_next:
    strb r3, [r1], #1   @ Store character as-is and move to next
    b final_check       @ Go to final check
```

1. **Επεξεργασία Χαρακτήρων:**
   - Φορτώνεται ο χαρακτήρας από τη συμβολοσειρά και αυξάνεται ο μετρητής χαρακτήρων.
   - Αν είναι χαρακτήρας νέας γραμμής, μεταφέρεται η εκτέλεση στην έξοδο.

2. **Αναγνώριση αριθμού:**
    - Αν ο χαρακτήρας είναι μικρότερος από τον κωδικό ASCII #48 (‘0’) τότε πάμε στον επόμενο χαρακτήρα.
    - Αν ο χαρακτήρας είναι μικρότερος ή ίσος από τον κωδικό ASCII #57 (‘9’) τότε κάνε την μετατροπή αριθμού με την `convert_digit`.
3. **Αναγνώριση κεφαλαίων:**
    - Αν ο χαρακτήρας είναι μικρότερος από τον κωδικό ASCII #65 (‘A’) τότε πάμε στον επόμενο χαρακτήρα.
    - Αν ο χαρακτήρας είναι μικρότερος ή ίσος από τον κωδικό ASCII #90 (‘Z’) τότε κάνε την μετατροπή από κεφαλαία σε πεζά με την `to_lowercase`.
4. **Αναγνώριση πεζών:**
    - Αν ο χαρακτήρας είναι μικρότερος από τον κωδικό ASCII #97 (‘a’) τότε πάμε στον επόμενο χαρακτήρα.
    - Αν ο χαρακτήρας είναι μικρότερος ή ίσος από τον κωδικό ASCII #122 (‘z’) τότε κάνε την μετατροπή από πεζά σε κεφαλαία με την `to_uppercase`.
5. **Άλλοι χαρακτήρες:**
    - Οι χαρακτήρες που δεν μετασχηματίζονται αποθηκεύονται όπως είναι στην συμβολοσειρά εξόδου και η μετατροπή συνεχίζεται.

```
convert_digit:
    cmp r3, #53         @ Check if digit < 5
    addlt r3, #5        @ Add 5 if less than 5
    subge r3, #5        @ Subtract 5 if >= 5
    strb r3, [r1], #1   @ Store transformed digit and move to next
    b final_check

to_uppercase:
    sub r3, r3, #32     @ Convert lowercase to uppercase
    strb r3, [r1], #1   @ Store transformed character and move to next
    b final_check

to_lowercase:
    add r3, r3, #32     @ Convert uppercase to lowercase
    strb r3, [r1], #1   @ Store transformed character and move to next
    b final_check

final_check:
    cmp r4, #32         @ Check if counter reached 32
    beq output          @ If 32 characters processed, go to output
    b convert           @ Otherwise, process the next character
```

1. **Μετατροπή Αριθμού:**
    - Αν ο αριθμός είναι από 0 έως 4 τότε προσθέτουμε 5 σε αυτόν.
    - Αν ο αριθμός είναι από 5 έως 9 τότε αφαιρούμε 5 από αυτόν.
2. **Μετατροπή από Πεζά σε Κεφαλαία:**
    - Αφαιρώντας 32 από τον κωδικό ASCII των πεζών τα μετατρέπουμε σε κεφαλαία.
3. **Μετατροπή από Κεφαλαία σε Πεζά:**
    - Προσθέτοντας 32 στον κωδικό ASCII των κεφαλαίων τα μετατρέπουμε με πεζά.
4. **Έλεγχος Μήκους:**
    - Αν έχουμε επεξεργαστεί 32 χαρακτήρες, πήγαινε στην έξοδο, αλλιώς συνέχισε.

```
output:
    cmp r4, #32
    strbeq r5, [r1, #-1] @ if 32nd character != '\n' make it '\n'
    mov r0, #1          @ stdout
    ldr r1, =input_str  @ Address of the string in memory
    mov r2, r4          @ Length of the string
    mov r7, #4          @ write system call
    swi 0
    cmp r4, #32
    beq flush_buffer
    b loop

flush_buffer:
    mov r7, #3          @ read system call
    mov r0, #2
    ldr r1,=input_str
    mov r2, #(1 << 30)  @ flush
    swi 0
    b loop

exit_program:
    pop {ip, pc}
    mov r7, #1          @ exit system call
    swi 0
```

1. **Εκτύπωση Εξόδου:**
    - Μετατροπή του τελευταίου χαρακτήρα της συμβολοσειράς σε `\n` και χρήση του `write` system call για την εκτύπωση της επεξεργασμένης συμβολοσειράς.
2. **Καθαρισμός buffer:**
    - Εκκαθάριση του buffer μετά την έξοδο και προετοιμασία για νέα είσοδο.
3. **Τερματισμός Προγράμματος:**
    - Επιστροφή από τη συνάρτηση και τερματισμός μέσω του system call `exit`.

Ανοίγουμε λοιπόν το qemu και συνδεόμαστε με ssh για πιο εύκολη πλοήγηση. Δημιουργούμε ένα ask1.s αρχείο και τοποθετούμε εκεί τον κώδικα μας. Επιπλέον, δημιουργούμε ένα Makefile για το compilation του προγράμματος:

```
CC = gcc
CFLAGS = -Wall  -g

ask1: ask1.s
    $(CC) $(CFLAGS) ask1.s -o ask1

clean:
    rm -f ask1
```

Τρέχουμε το πρόγραμμά μας και παίρνουμε την παρακάτω έξοδο:

```
root@debian-armel:/home/user/ask1# ls
ask1.s  Makefile
root@debian-armel:/home/user/ask1# make
gcc -Wall  -g ask1.s -o ask1
root@debian-armel:/home/user/ask1# ./ask1
Input a string of up to 32 characters (Terminate with Q or q): STRING123789
string678234
Input a string of up to 32 characters (Terminate with Q or q): string4561290
STRING9016745
Input a string of up to 32 characters (Terminate with Q or q): StRiNg&%#$1289
sTrInG&%#$6734
Input a string of up to 32 characters (Terminate with Q or q): STOPq
stopQ
Input a string of up to 32 characters (Terminate with Q or q): stopQ
STOPq
Input a string of up to 32 characters (Terminate with Q or q): Q
root@debian-armel:/home/user/ask1#
```

Παρατηρούμε ότι πράγματι το πρόγραμμα εκτελεί σωστά την λειτουγία του.

Ερώτημα 2:

Επικοινωνία των guest και host μηχανημάτων μέσω σειριακής θύρας.

Σκοπός της δεύτερης άσκησης είναι η επικοινωνία μεταξύ δύο μηχανημάτων (host και guest) μέσω εικονικής σειριακής θύρας. Το πρόγραμμα που τρέχει στο **host** λαμβάνει ως είσοδο μια συμβολοσειρά μεγέθους έως 64 χαρακτήρων, την αποστέλλει μέσω της σειριακής θύρας στο **guest**, και λαμβάνει ως απάντηση τον χαρακτήρα με τη μεγαλύτερη συχνότητα εμφάνισης και τον αριθμό εμφανίσεών του.

Ο χαρακτήρας του κενού εξαιρείται από την καταμέτρηση, και σε περίπτωση ισοβαθμίας επιλέγεται ο χαρακτήρας με τον μικρότερο ASCII κώδικα.

Η επικοινωνία βασίζεται στη χρήση της βιβλιοθήκης **termios** για την παραμετροποίηση της σειριακής θύρας. Στο περιβάλλον **Linux**, η χρήση pseudo-terminals (PTS) επιτρέπει τη σύνδεση των δύο συστημάτων:

- Το πρόγραμμα στο **host** ανοίγει τη συσκευή `/dev/pts/x` που δημιουργείται από το QEMU.
- Το **guest** χρησιμοποιεί τη σειριακή θύρα `ttyAMA0` για τη σύνδεση.

Ας διαχωρίσουμε το πρόγραμμα **host** σε μικρότερα τμήματα για να εξηγήσουμε κάθε μέρος με λεπτομέρεια.

```
if (argc != 2) {
    fprintf(stderr, "Usage: %s <device>\nExample: %s /dev/pts/7\n",
argv[0], argv[0]);
    exit(EXIT_FAILURE);
}

printf("Type a message to send to the guest:\n");
bytesRead = read(STDIN_FILENO, sendBuffer, MAX_SIZE);
if (bytesRead < 0) {
    perror("Error reading from stdin");
    exit(EXIT_FAILURE);
}

if (bytesRead == MAX_SIZE) {
    fprintf(stderr, "Error: Input exceeds 64 characters.\n");
    fflush(stdout);
    exit(EXIT_FAILURE);
} else {
    printf("Input accepted: %zd bytes.\n", bytesRead);
}
```

1. **Έλεγχος παραμέτρων:**
    - Το πρόγραμμα απαιτεί ένα όρισμα που να δείχνει τη συσκευή της σειριακής θύρας (π.χ. `/dev/pts/7`).
    - Αν δεν δοθεί, εμφανίζεται μήνυμα χρήσης και το πρόγραμμα τερματίζει.
2. **Ανάγνωση συμβολοσειράς από το χρήστη:**

- Η `read` διαβάζει έως 65 χαρακτήρες από το standard input (stdin).
- Ελέγχεται αν το μήκος υπερβαίνει τους 64 χαρακτήρες. Σε τέτοια περίπτωση, εμφανίζεται σφάλμα και το πρόγραμμα τερματίζει.

3. **Ενημέρωση χρήστη:**
   - Αν η είσοδος είναι έγκυρη, εμφανίζεται μήνυμα με τον αριθμό των χαρακτήρων που εισήχθησαν.

```
serial_fd = open(argv[1], O_RDWR | O_NOCTTY);
if (serial_fd < 0) {
    perror("Failed to open the specified device");
    return EXIT_FAILURE;
}
```

1. **Άνοιγμα της σειριακής θύρας:**
   - Το πρόγραμμα ανοίγει τη συσκευή που παρέχεται ως όρισμα (π.χ. `/dev/pts/7`) με δικαιώματα ανάγνωσης και εγγραφής (`O_RDWR`).
   - Η επιλογή `O_NOCTTY` διασφαλίζει ότι η συσκευή δεν γίνεται το controlling terminal της διεργασίας.
2. **Διαχείριση σφαλμάτων:**
   - Αν η `open` αποτύχει (π.χ., αν η συσκευή δεν υπάρχει ή δεν έχει άδεια), εμφανίζεται μήνυμα σφάλματος μέσω της `perror` και το πρόγραμμα τερματίζει.

```
if (tcgetattr(serial_fd, &ttyAttributes) < 0) {
    fprintf(stderr, "Unable to get terminal attributes.\n");
    close(serial_fd);
    return EXIT_FAILURE;
}

ttyAttributes.c_lflag = ICANON;
ttyAttributes.c_cflag |= (CLOCAL | CREAD | CS8);
ttyAttributes.c_cflag &= ~CSTOPB;
ttyAttributes.c_cc[VMIN] = 1;
ttyAttributes.c_cc[VTIME] = 0;

if (cfsetispeed(&ttyAttributes, B9600) < 0 ||
cfsetospeed(&ttyAttributes, B9600) < 0) {
    fprintf(stderr, "Failed to configure baud rate.\n");
    close(serial_fd);
    return EXIT_FAILURE;
```

```
}

if (tcsetattr(serial_fd, TCSANOW, &ttyAttributes) < 0) {
    fprintf(stderr, "Could not apply terminal settings.\n");
    close(serial_fd);
    return EXIT_FAILURE;
}

tcflush(serial_fd, TCIOFLUSH);
```

1. **Ανάγνωση τρέχουσας διαμόρφωσης θύρας:**
    - Η `tcgetattr` λαμβάνει τις τρέχουσες ρυθμίσεις της σειριακής θύρας και τις αποθηκεύει στη δομή `ttyAttributes`.
    - Αν αποτύχει, εμφανίζεται σφάλμα και η θύρα κλείνει.
2. **Ορισμός ρυθμίσεων θύρας:**
    - **Canonical mode:** Με τη χρήση `ICANON`, οι χαρακτήρες διαβάζονται ως γραμμές (τερματίζονται με EOL).
    - **8-N-1 format:**
        - Το `CS8` ορίζει 8 data bits.
        - Το `CLOCAL` αποτρέπει την αναμονή για signal της σειριακής θύρας.
        - Το `CREAD` ενεργοποιεί τη δυνατότητα ανάγνωσης δεδομένων.
        - Το `~CSTOPB` ορίζει 1 stop bit.
    - **Baud rate:** Ορίζεται στα 9600 bits/second.
3. **Εφαρμογή των ρυθμίσεων:**
    - Η `tcsetattr` εφαρμόζει τις αλλαγές.
    - Αν αποτύχει, εμφανίζεται σφάλμα και η θύρα κλείνει.
4. **Καθαρισμός buffer:**
    - Με την `tcflush` απορρίπτονται δεδομένα που εκκρεμούν για ανάγνωση ή εγγραφή.

```
printf("Sending data to guest...\n");
if (write(serial_fd, sendBuffer, MAX_SIZE) < 0) {
    perror("Error writing to device");
    close(serial_fd);
    return EXIT_FAILURE;
}
```

1. **Αποστολή συμβολοσειράς:**
    - Η `write` στέλνει τα δεδομένα που εισήγαγε ο χρήστης μέσω της σειριακής θύρας.
    - Γίνεται προσπάθεια αποστολής του buffer μεγέθους `MAX_SIZE` (65 bytes).
2. **Διαχείριση σφαλμάτων:**
    - Αν η `write` αποτύχει, εμφανίζεται μήνυμα σφάλματος και η θύρα κλείνει.

```
printf("Awaiting response from guest...\n");
if (read(serial_fd, recvBuffer, MAX_SIZE) < 0) {
    perror("Error reading from device");
    close(serial_fd);
    return EXIT_FAILURE;
}
```

1. **Αναμονή απάντησης:**
    - Η `read` διαβάζει δεδομένα από τη σειριακή θύρα στο `recvBuffer`.
2. **Διαχείριση σφαλμάτων:**
    - Αν η `read` αποτύχει, εμφανίζεται σφάλμα και η θύρα κλείνει.

```
int frequency = recvBuffer[2] - '0';
if (frequency > 0) {
    printf("Most frequent character: '%c' - Occurrences: %d\n",
recvBuffer[0], frequency);
} else {
    printf("No valid characters received or processed.\n");
}
```

1. **Αποκωδικοποίηση απάντησης:**
    - Ο πρώτος χαρακτήρας (`recvBuffer[0]`) είναι ο πιο συχνός.
    - Ο τρίτος χαρακτήρας (`recvBuffer[2]`) περιέχει τη συχνότητα, κωδικοποιημένη ως ASCII. Η αφαίρεση του `'0'` μετατρέπει την τιμή σε ακέραιο.
2. **Εμφάνιση αποτελεσμάτων:**
    - Αν η συχνότητα είναι μεγαλύτερη από 0, εμφανίζονται τα αποτελέσματα.
    - Διαφορετικά, εμφανίζεται μήνυμα ότι δεν ελήφθησαν έγκυροι χαρακτήρες.

```
close(serial_fd);
return EXIT_SUCCESS;
```

1. **Κλείσιμο της θύρας:**
    - Η `close` απελευθερώνει τους πόρους που συνδέονται με τη σειριακή θύρα.
2. **Επιτυχής έξοδος:**
    - Το πρόγραμμα τερματίζει με κωδικό επιτυχίας (`EXIT_SUCCESS`).

Θα επαναλάβουμε την ίδια διαδικασία για το πρόγραμμα **guest**.

```
    device_open:
    ldr r0, =dev_path           @ r0 = διαδρομή συσκευής
("/dev/ttyAMA0")
    ldr r1, =#258               @ O_RDWR | O_NOCTTY
    mov r7, #SYSCALL_OPEN       @ αριθμός κλήσης open
    swi 0                       @ κλήση συστήματος για open
    mov r6, r0                  @ αποθήκευση του file descriptor στο
r6
```

- Η κλήση συστήματος `open` ανοίγει τη σειριακή θύρα στη διαδρομή `/dev/ttyAMA0`.
- Flags:
    - `O_RDWR`: Επιτρέπει ανάγνωση και εγγραφή.
    - `O_NOCTTY`: Αποτρέπει τη συσκευή από το να γίνει το controlling terminal.
- Ο file descriptor (fd) που επιστρέφεται αποθηκεύεται στο `r6` για μελλοντική χρήση.

```
setup_terminal:
    mov r0, r6                  @ r0 = file descriptor
    ldr r2, =term_config        @ r2 = δείκτης στη δομή termios
    mov r1, #0                  @ r1 = TCSANOW (άμεση εφαρμογή
ρυθμίσεων)
    bl tcsetattr                @ κλήση της συνάρτησης tcsetattr
```

- Η συνάρτηση `tcsetattr` εφαρμόζει τις ρυθμίσεις που περιέχονται στη δομή `term_config` στη σειριακή θύρα.
- Οι ρυθμίσεις περιλαμβάνουν:
    - Canonical mode: Επεξεργασία εισόδου ανά γραμμή.
    - Ταχύτητα μετάδοσης (baud rate): 9600.

```
read_data:
    mov r0, r6                  @ file descriptor
    ldr r1, =input_data         @ buffer εισόδου
    mov r2, #64                 @ μέγεθος δεδομένων προς ανάγνωση
    mov r7, #SYSCALL_READ       @ αριθμός κλήσης read
    swi 0                       @ κλήση συστήματος για ανάγνωση
    ldr r0, =input_data         @ r0 δείκτης στην αρχή του buffer
```

- Διαβάζουμε έως 64 χαρακτήρες από τη σειριακή θύρα χρησιμοποιώντας το syscall `read`.
- Τα δεδομένα αποθηκεύονται στον buffer `input_data`.

```
analyze_string:
    ldr r3, =freq_table         @ βάση πίνακα συχνοτήτων
    mov r1, #0                  @ δείκτης για ανάγνωση χαρακτήρων

char_loop:
    ldrb r2, [r0], #1           @ φόρτωση χαρακτήρα και αύξηση δείκτη
    cmp r2, #10                 @ έλεγχος αν είναι newline
    beq done_analyze            @ τερματισμός αν είναι newline
    ldrb r4, [r3, r2]           @ φόρτωση τρέχουσας συχνότητας για
τον χαρακτήρα
    add r4, r4, #1              @ αύξηση συχνότητας
    strb r4, [r3, r2]           @ αποθήκευση ενημερωμένης συχνότητας
    b char_loop                 @ επανάληψη για τον επόμενο χαρακτήρα
```

- Σαρώνουμε τη συμβολοσειρά από τον buffer `input_data`.
- Για κάθε χαρακτήρα, αυξάνουμε τη συχνότητά του στον πίνακα `freq_table`.
- Η διαδικασία τερματίζεται όταν συναντηθεί ο χαρακτήρας αλλαγής γραμμής (`newline`).

```
done_analyze:
    mov r1, #0                  @ αρχικοποίηση δείκτη
    mov r0, #0                  @ r0 = μέγιστη συχνότητα
    ldr r3, =freq_table         @ βάση πίνακα συχνοτήτων
    strb r0, [r3, #32]          @ μηδενισμός συχνότητας για τον
χαρακτήρα space

freq_check:
    ldrb r2, [r3, r1]           @ συχνότητα στο index r1
    cmp r0, r2                  @ σύγκριση με τρέχουσα μέγιστη
συχνότητα
    movlt r4, r1                @ αν είναι μεγαλύτερη, ενημέρωση
χαρακτήρα
    movlt r0, r2                @ ενημέρωση μέγιστης συχνότητας
    add r1, r1, #1              @ επόμενος δείκτης
    cmp r1, #256                @ τέλος πίνακα ASCII;
    beq freq_done
    b freq_check
```

- Σαρώνουμε τον πίνακα συχνοτήτων (`freq_table`) για να εντοπίσουμε τον χαρακτήρα με τη μέγιστη συχνότητα.
- Αν δύο χαρακτήρες έχουν την ίδια συχνότητα, επιλέγεται ο χαρακτήρας με τον μικρότερο ASCII κώδικα.
- Αγνοεί τη συχνότητα του κενό χαρακτήρα (`space`).

```
freq_done:
    add r0, r0, #48             @ μετατροπή συχνότητας σε ASCII ('0'
+ freq)
    ldr r3, =output_data        @ buffer αποτελέσματος
    strb r4, [r3]               @ output_data[0] = χαρακτήρας
    strb r0, [r3, #2]           @ output_data[2] = συχνότητα ως ASCII
```

- Αποθηκεύουμε τον χαρακτήρα με τη μέγιστη συχνότητα στο `output_data[0]`.
- Μετατρέπουμε τη συχνότητα σε ASCII και την αποθηκεύει στο `output_data[2]`.

```
write_result:
    mov r0, r6                  @ file descriptor
    ldr r1, =output_data        @ buffer αποτελέσματος
    ldr r2, =out_len            @ μήκος δεδομένων
    mov r7, #SYSCALL_WRITE      @ αριθμός κλήσης write
    swi 0                       @ κλήση write
```

- Στέλνουμε το αποτέλεσμα στον host μέσω της σειριακής θύρας.
- Το αποτέλεσμα περιλαμβάνει τον πιο συχνό χαρακτήρα και τη συχνότητά του.

```
close_device:
    mov r0, r6                  @ file descriptor
    mov r7, #SYSCALL_CLOSE      @ αριθμός κλήσης close
    swi 0                       @ κλήση close
```

```
exit_program:
    mov r0, #0                  @ κωδικός εξόδου
    mov r7, #SYSCALL_EXIT       @ αριθμός κλήσης exit
    swi 0                       @ κλήση exit
```

- Κλείνουμε τη σειριακή θύρα απελευθερώνοντας τους πόρους που χρησιμοποιήθηκαν.
- Τερματίζουμε το πρόγραμμα επιστρέφοντας τον κωδικό εξόδου `0`.

`term_config`

Η δομή `term_config` περιέχει τις παραμέτρους της σειριακής θύρας, οι οποίες ορίζονται σύμφωνα με τις απαιτήσεις της άσκησης. Περιλαμβάνει:

- **Ταχύτητα μετάδοσης (baud rate):** Ορίζεται στα 9600 bits/second.
- **Λειτουργία:** Canonical mode (επεξεργασία εισόδου ανά γραμμή), 8 data bits, χωρίς parity, 1 stop bit (8-N-1).
- **Αρχικοποιήσεις μηδενισμού:** Όλα τα πεδία που δεν χρησιμοποιούνται έχουν τιμή `0`.

`freq_table`

Ο πίνακας `freq_table` αποτελεί έναν πίνακα συχνοτήτων μεγέθους 256 θέσεων, όπου:

- Κάθε θέση αντιστοιχεί σε έναν ASCII χαρακτήρα.
- Οι συχνότητες των χαρακτήρων αρχικοποιούνται σε `0`.
- Χρησιμοποιείται για την καταγραφή των εμφανίσεων κάθε χαρακτήρα κατά την ανάλυση της συμβολοσειράς.

Οι αριθμοί των syscall που χρησιμοποιούνται στο πρόγραμμα είναι:

- **SYSCALL_OPEN**: 5 – Χρησιμοποιείται για το άνοιγμα της σειριακής θύρας.
- **SYSCALL_READ**: 3 – Χρησιμοποιείται για την ανάγνωση δεδομένων από τη σειριακή θύρα.
- **SYSCALL_WRITE**: 4 – Χρησιμοποιείται για την αποστολή δεδομένων στον host.
- **SYSCALL_CLOSE**: 6 – Χρησιμοποιείται για το κλείσιμο της σειριακής θύρας.
- **SYSCALL_EXIT**: 1 – Χρησιμοποιείται για τον τερματισμό του προγράμματος.

Για το setup του guest μηχανήματος εκτελούμε την παρακάτω εντολή:

```
zoegken@zoegken-VirtualBox:~$ cd qemu_arm_vm/
zoegken@zoegken-VirtualBox:~/qemu_arm_vm$ sudo qemu-system-arm -M versatilepb -kernel vmlinuz-3.2.0-4-versatile -initrd initrd.img-3
.2.0-4-versatile -hda debian_wheezy_armel_standard.qcow2 -append "root=/dev/sda1" -net nic -net user,hostfwd=tcp:127.0.0.1:22223-:22
 -serial pty
char device redirected to /dev/pts/4 (label serial0)
```

Το qemu δημιουργεί το αρχείου στο /dev/pts/ και μας γράφει στο command line ποιο ακριβώς αρχείο είναι αυτό. Ανοίγουμε λοιπόν το qemu και συνδεόμαστε με ssh για πιο εύκολη πλοήγηση. Δημιουργούμε ένα guest.s αρχείο και τοποθετούμε τον κώδικα μας. Επιπλέον δημιουργούμε ένα Makefile για το compilation του προγράμματος:

```
CC = gcc
CFLAGS = -Wall  -g

guest: guest.s
	$(CC) $(CFLAGS) guest.s -o guest

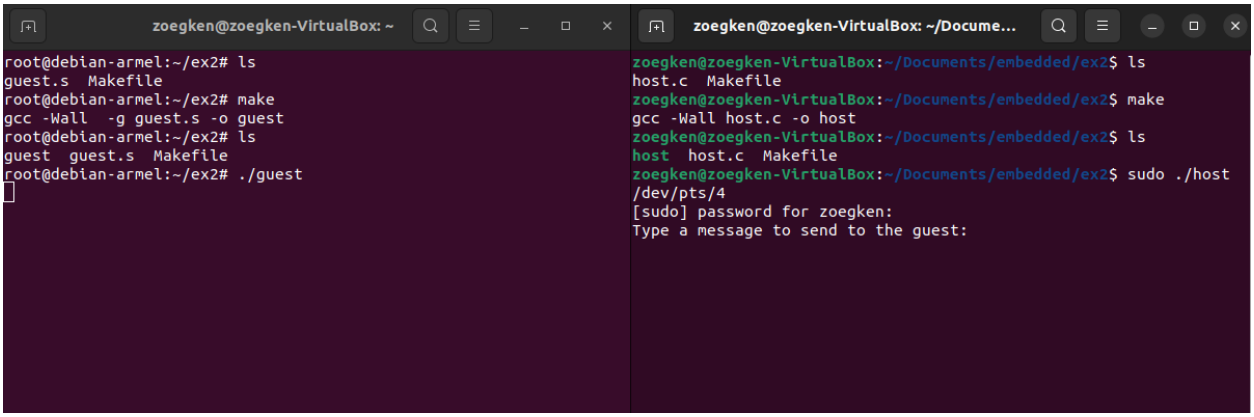
clean:
	rm -f guest
```

Από την άλλη για το host πρόγραμμα δημιουργούμε πάλι ένα host.c αρχείο σε ένα άλλο τερματικό και το αντίστοιχο Makefile:

```
CC = gcc
CFLAGS = -Wall
```

```
host: host.c
	$(CC) $(CFLAGS) host.c -o host

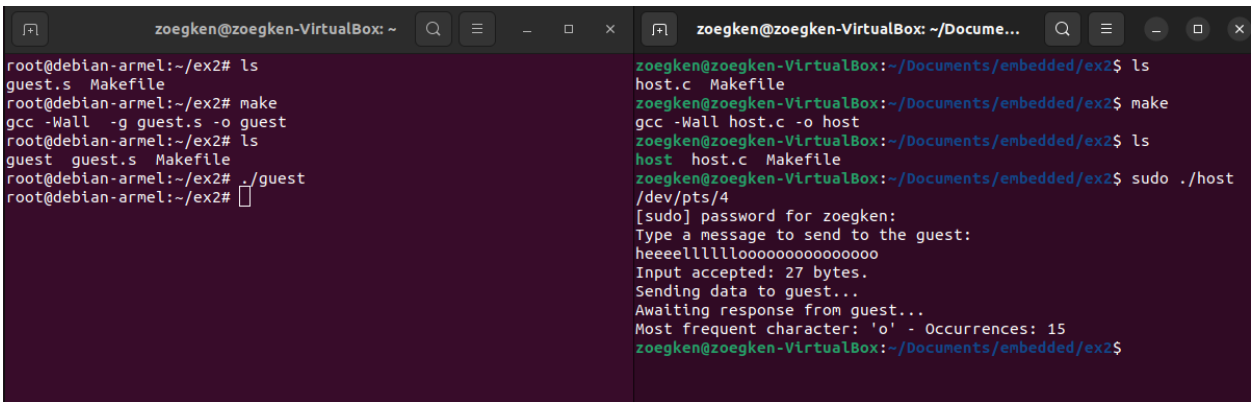
clean:
	rm -f host
```



Τρέχουμε και τα δύο προγράμματα παράλληλα. Στην περίπτωση του host, για να τρέξουμε το πρόγραμμα εκτελούμε την εντολή:

```
sudo ./host /dev/pts/4
```

Χρειάζεται να χρησιμοποιήσουμε το sudo για να εκτελέσουμε το πρόγραμμα `./host` με αυξημένα δικαιώματα, επειδή η πρόσβαση σε συγκεκριμένους πόρους σε επίπεδο συστήματος, όπως ψευδοτερματικά (/dev/pts/4), απαιτεί συχνά δικαιώματα διαχείρισης. Η εκτέλεση του προγράμματος ως sudo διασφαλίζει ότι έχουμε τα απαραίτητα δικαιώματα για άνοιγμα, ανάγνωση και εγγραφή στο αρχείο της συσκευής.



Παρατηρούμε ότι πράγματι το κάθε πρόγραμμα εκτελεί σωστά την λειτουργία του.

Ερώτημα 3:

Σύνδεση κώδικα C με κώδικα assembly του επεξεργαστή ARM.

Σκοπός της τρίτης άσκησης είναι ο συνδυασμός κώδικα που είναι γραμμένος σε C μαζί με συναρτήσεις που είναι γραμμένες σε assembly του επεξεργαστή ARM. Σε ένα αρχείο με όνομα `string_manipulation.c` υλοποιείται μια σειρά λειτουργιών πάνω σε ένα αρχείο εισόδου που αποτελείται από 512 γραμμές. Κάθε γραμμή περιέχει μια τυχαία συμβολοσειρά με μήκος από 8 έως 64 χαρακτήρες.

Κατά την εκτέλεση του προγράμματος, παράγονται τρία αρχεία εξόδου με τις εξής πληροφορίες:

1. Το πρώτο περιέχει το μήκος της κάθε γραμμής του αρχείου εισόδου.
2. Το δεύτερο περιέχει τις συμβολοσειρές του αρχείου εισόδου ενωμένες (concatenated) ανά 2.
3. Το τρίτο περιέχει τις συμβολοσειρές του αρχείου εισόδου ταξινομημένες σε αύξουσα αλφαβητική σειρά.

Για την επίτευξη των παραπάνω λειτουργιών, το πρόγραμμα χρησιμοποιεί τις συναρτήσεις **strlen**, **strcpy**, **strcat**, και **strcmp** από τη βιβλιοθήκη **string.h**. Αυτό που ζητείται από εμάς είναι να αντικαταστήσουμε τις παραπάνω συναρτήσεις με δικές μας γραμμένες αποκλειστικά σε ARM assembly. Όσο για τον τρόπο σύνδεσης assembly και γλώσσας C, θα υλοποιήσουμε κάθε μία από τις ζητούμενες συναρτήσεις σε ξεχωριστά αρχεία και θα τις δηλώσουμε ως **extern** στον C κώδικα του αρχείου `string_manipulation.c`.

strlen.s

Η συνάρτηση δέχεται ως είσοδο τη διεύθυνση της συμβολοσειράς μέσω του καταχωρητή r0. Αρχικά, αρχικοποιεί τον μετρητή μήκους στον καταχωρητή r3 με την τιμή 0. Στη συνέχεια, μέσω ενός βρόχου, διαβάζει κάθε χαρακτήρα της συμβολοσειράς, ελέγχοντας αν είναι το τερματικό σύμβολο (null terminator). Αν βρει το τερματικό σύμβολο, τερματίζει τον βρόχο και επιστρέφει το μήκος της συμβολοσειράς αποθηκεύοντας το στον καταχωρητή r0. Αν δεν είναι το τερματικό σύμβολο, αυξάνει τον μετρητή και συνεχίζει με τον επόμενο χαρακτήρα.

```
.text
.align 4
.global strlen
.type strlen, %function

@ Function input:
@ r0: address of the string (str)
strlen:
```

```
    mov r3, #0          @ Initialize the length counter in r3
length_loop:
    ldrb r2, [r0], #1   @ Read the next character and advance the pointer
    cmp r2, #0          @ Check if the current character is the null terminator
    beq finish          @ If null terminator found, exit the loop
    add r3, r3, #1      @ Otherwise, increment the length counter
    b length_loop       @ Repeat for the next character
finish:
    mov r0, r3          @ Store the computed length in r0 as the return value
    bx lr               @ Return to the caller
```

strcpy.s

Η συνάρτηση δέχεται δύο παραμέτρους: την διεύθυνση του προορισμού (dst) μέσω του καταχωρητή r0 και τη διεύθυνση της πηγής (src) μέσω του r1. Αρχικά, αποθηκεύει τη διεύθυνση του προορισμού στον καταχωρητή r4. Στη συνέχεια, μέσω ενός βρόχου, διαβάζει byte προς byte από τη συμβολοσειρά πηγής και τα αντιγράφει στον προορισμό. Ελέγχει αν το byte είναι το τερματικό σύμβολο (null terminator) και αν δεν είναι, συνεχίζει να αντιγράφει τον επόμενο χαρακτήρα. Όταν φτάσει στο τερματικό σύμβολο, επιστρέφει τη διεύθυνση του προορισμού μέσω του r0 και ολοκληρώνει τη συνάρτηση.

```
.text
.align 4
.global strcpy
.type strcpy, %function

@ Function inputs:
@ r0: destination buffer (dst)
@ r1: source string (src)
strcpy:
    mov r4, r0          @ Save the starting address of the destination buffer
in r4
copy_loop:
    ldrb r3, [r1], #1   @ Read the next byte from the source string and move to
the next position
    strb r3, [r0], #1   @ Write the byte to the destination buffer and move to
the next position
    cmp r3, #0          @ Check if the byte is the null terminator
    bne copy_loop       @ If not, keep copying the next byte
finish:
    mov r0, r4          @ Restore the original destination address to r0
(return value)
    bx lr               @ Exit the function and return to the caller
```

strcat.s

Η συνάρτηση δέχεται δύο παραμέτρους: την διεύθυνση της συμβολοσειράς προορισμού (dst) μέσω του καταχωρητή r0 και τη διεύθυνση της συμβολοσειράς πηγής (src) μέσω του r1. Αρχικά, αποθηκεύει τη διεύθυνση του προορισμού στον καταχωρητή r5. Στη συνέχεια, μέσω ενός βρόχου, βρίσκει το τέλος της συμβολοσειράς προορισμού (αναζητώντας το τερματικό σύμβολο). Όταν το βρει, τοποθετεί τον δείκτη στην κατάλληλη θέση για να αντικαταστήσει το τερματικό σύμβολο με την πηγή. Στη συνέχεια, αντιγράφει byte προς byte τη συμβολοσειρά πηγής στο τέλος της προορισμένης συμβολοσειράς μέχρι να φτάσει στο τερματικό σύμβολο της πηγής. Όταν ολοκληρωθεί η αντιγραφή, επιστρέφει τη διεύθυνση του προορισμού μέσω του r0 και ολοκληρώνει τη συνάρτηση.

```
.text
.align 4
.global strcat
.type strcat, %function

@ Function inputs:
@ r0: destination string (dst)
@ r1: source string (src)
strcat:
    mov r5, r0          @ Save the initial destination address in r5
find_end:
    ldrb r3, [r0], #1   @ Load a byte from the destination string and advance
the pointer
    cmp r3, #0          @ Check if it's the null terminator
    bne find_end        @ Continue until the end of the destination string is
reached
    sub r0, r0, #1      @ Step back to overwrite the null terminator with the
source string
append_src:
    ldrb r4, [r1], #1   @ Load a byte from the source string and advance the
source pointer
    strb r4, [r0], #1   @ Write the byte to the current position in the
destination string
    cmp r4, #0          @ Check if it's the null terminator of the source
string
    bne append_src      @ Continue until the source string ends
finish:
    mov r0, r5          @ Restore the original destination address to r0
    bx lr               @ Return to the caller
```

strcmp.s

Η συνάρτηση δέχεται δύο παραμέτρους: τη διεύθυνση της πρώτης συμβολοσειράς (str1) μέσω του καταχωρητή r0 και τη διεύθυνση της δεύτερης συμβολοσειράς (str2) μέσω του r1. Αρχικά, ορίζεται το αποτέλεσμα σε 0, θεωρώντας ότι οι συμβολοσειρές είναι ίδιες. Μέσα σε έναν βρόχο, συγκρίνει χαρακτήρα προς χαρακτήρα τις δύο συμβολοσειρές. Αν βρει χαρακτήρες που διαφέρουν, καθορίζει ποια συμβολοσειρά είναι μικρότερη ή μεγαλύτερη και αποθηκεύει το αποτέλεσμα (-1 ή 1) στον καταχωρητή r5. Αν οι χαρακτήρες είναι ίδιοι και δεν έχει φτάσει στο τέλος κάποιας από τις δύο συμβολοσειρές, συνεχίζει τη σύγκριση. Αν φτάσουν και οι δύο συμβολοσειρές στο τερματικό σύμβολο ταυτόχρονα, επιστρέφει 0, υποδεικνύοντας ότι είναι ίδιες. Στο τέλος, το αποτέλεσμα επιστρέφεται μέσω του καταχωρητή r0 και η συνάρτηση ολοκληρώνεται.

```
.text
.align 4
.global strcmp
.type strcmp, %function

@ Function inputs:
@ r0: address of the first string (str1)
@ r1: address of the second string (str2)
strcmp:
    mov r5, #0          @ Default result is 0 (strings are equal)
compare_loop:
    ldrb r3, [r0], #1   @ Load a character from str1 and advance pointer
    ldrb r4, [r1], #1   @ Load a character from str2 and advance pointer
    cmp r3, r4          @ Compare the characters from str1 and str2
    bne determine_order @ If not equal, decide which string is
lexicographically smaller
    cmp r3, #0          @ Check if we've reached the end of both strings
    beq finish          @ If both ended simultaneously, strings are equal
    b compare_loop      @ Otherwise, continue comparing the next characters
determine_order:
    movlt r5, #-1       @ If str1 < str2, set result to -1
    movgt r5, #1        @ If str1 > str2, set result to 1
finish:
    mov r0, r5          @ Place the result into r0 for returning
    bx lr               @ Return to the caller
```

Για να ενσωματώσουμε τις παραπάνω συναρτήσεις στο αρχείο `string_manipulation.c` και να αντικαταστήσουμε τις αντίστοιχες δηλώσεις της βιβλιοθήκης **string.h**, προσθέτουμε τις εξής γραμμές κώδικα:

```c
// string.h removed since functions are defined by user
// #include <string.h>

// define all fucntions to be replaced as extern
extern size_t strlen(const char *str);
extern char *strcpy(char *dst, const char *src);
extern char *strcat(char *dst, const char *src);
extern int strcmp(const char *str1, const char *str2);
// end of defintion
```

Για τη μεταγλώττιση και σύνδεση των παραπάνω αρχείων, χρησιμοποιήσαμε το Makefile που παρατίθεται παρακάτω. Συγκεκριμένα, πρώτα μεταγλωττίζουμε τον κώδικα των τεσσάρων συναρτήσεων με την επιλογή -c για να μην συνδεθούνε. Στη συνέχεια, μεταγλωττίζουμε με τον ίδιο τρόπο και το αρχείο string_manipulation.c. Τέλος, συνδέουμε όλα τα αρχεία για να παραχθεί το εκτελέσιμο string_manipulation.out.

```
CC = gcc
CFLAGS = -Wall -g
OBJ = string_manipulation.o strcmp.o strcpy.o strcat.o strlen.o

all: string_manipulation.out

string_manipulation.o: string_manipulation.c
    $(CC) -c -o $@ $^ $(CFLAGS)

%.o: %.s
    $(CC) -c -o $@ $< $(CFLAGS)

string_manipulation.out: $(OBJ)
    $(CC) -o $@ $^ $(CFLAGS)

clean:
    rm -f *.o *.out
```

Ανοίγουμε λοιπόν το qemu και συνδεόμαστε με ssh για πιο εύκολη πλοήγηση. Δημιουργούμε ένα ask3 directory και τοποθετούμε εκεί όλα τα απαραίτητα αρχεία. Εκτελούμε την εντολή **make** και έπειτα τρέχουμε το εκτελέσιμο αρχείο που προκύπτει με την παρακάτω εντολή:

```
./string_manipulation.out rand_str_input_first.txt
```

Συγκρίνουμε τα τρία αρχεία που προέκυψαν από την παραπάνω υλοποίηση με τα αντίστοιχα τρία αρχεία που προκύπτουν αν εκτελέσουμε τον κώδικα με την βιβλιοθήκη string.h. Και στις δύο περιπτώσεις παίρνουμε τα ίδια αποτελέσματα, επομένως η υλοποίησή μας είναι σωστή.